frameworks. In addition, use the standard log levels of INFO, WARN, ERROR, and DEBUG so you can set your logs to output the data you need when you need it.

- ***INFO*** —Use this when you want to send information back to your logs.
- ***WARN*** —Use this when something isn't going well but isn't blowing up yet.
- ***ERROR*** —Something broke; always pay attention to these.
- ***DEBUG*** —Typically you don't look at DEBUG logs in a production environment unless you're trying to find a problem you can't figure out from the rest of your logs. Of course, this is great to have in nonproduction environments.

Pay attention to what your logs are telling you proactively

Although this may sound obvious, I've seen teams completely ignore their logs until the consumer was reporting something they couldn't reproduce. By that time it was too late; there were so many errors in the logs that trying to find the root of the problem became like searching for a needle in a haystack. Be a good developer and write data to your logs; then be a good developer and pay attention to what your logs tell you.

Use formats that are easy to work with

To be able to analyze the data in your logs, you'll want to make sure you're writing them in a format that's easy to parse, search, and aggregate. The following guidelines give details on how to do this:

- ***Log events in text instead of binary format.*** If you log things in binary, they're going to require decoding before you can do anything with them.
- ***Make the logs easy for humans to read.*** Your logs are essentially auditing everything your system does. If you want to make sense of that later, you had better be able to read what your logs are telling you.
- ***Use JSON, not XML.*** JSON is easy for developers to work with, easier to index, and a more human-readable format. You can use